EECE 306 Lab 7 notebook. Writing Your Own Poisson Solver

Table of Contents

Setup 1
Interpretation 7
Full test suite 8

Team 3. Carissa McWilliams, Mitch Nazareth, and Amelia Harris

Setup

```
clear; close all;
e0 = em.const.eps0();

% The Manufactured Solution
% The exact potential is chosen first, the source that produces it is
% derived, and the solver must give the potential back.
Vex = @(X,Y) sin(pi*X) .* sin(pi*Y);
rho = em.test.mms2d(Vex, 1);
G = em.solve.grid2d([0 1], [0 1], 81, 81);
mask = false(81,81); mask(1,:)=true; mask(end,:)=true; mask(:,1)=true;
mask(:,end)=true;
Vfix = zeros(81,81);
[Vs, info] = em.solve.poisson2d(G, rho, 1, mask, Vfix,
struct('method','direct'));
err81 = max(abs(Vs(:) - Vex(G.X(:), G.Y(:))));
fprintf('MMS max error on the 81 x 81 grid = %.3e\n', err81);

% Convergence under grid refinement
Ns = [21 41 81 161]';
errs = zeros(size(Ns));
for j = 1:numel(Ns)
    n = Ns(j);
    Gn = em.solve.grid2d([0 1], [0 1], n, n);
    mk = false(n,n); mk(1,:)=true; mk(end,:)=true; mk(:,1)=true;
mk(:,end)=true;
    Vn = em.solve.poisson2d(Gn, rho, 1, mk, zeros(n,n),
struct('method','direct'));
    errs(j) = max(abs(Vn(:) - Vex(Gn.X(:), Gn.Y(:))));
    fprintf('N = %3d   max error = %.3e\n', n, errs(j));
end
opts = struct('plot', false);
p = em.test.convergence(@(N) errs(Ns == N), Ns, opts);
fprintf('observed order p = %.2f  (expect near 2 for the five point stencil)
\n', p);

% Three iterations and a direct solve
```

```
n = 41;
Gn = em.solve.grid2d([0 1], [0 1], n, n);
mk = false(n,n); mk(1,:)=true; mk(end,:)=true; mk(:,1)=true; mk(:,end)=true;
Vf = zeros(n,n);
% A point by point sweep in an interpreted language is slow, and Jacobi
% needs thousands of them. It is therefore cut off at 3000 iterations on
% purpose. The residual histories tell the story long before then.
methods = {'jacobi', 'gs', 'sor'};
maxits  = [3000 10000 10000];
figure; hold on
for k = 1:3
    o = struct('method', methods{k}, 'tol', 1e-6, 'maxit', maxits(k));
    if strcmp(methods{k}, 'sor'), o.omega = 2/(1+sin(pi/(n-1))); end
    [Vk, ik] = em.solve.poisson2d(Gn, rho, 1, mk, Vf, o);
    semilogy(ik.resHist)
    fprintf('%-7s iterations = %5d\n', methods{k}, ik.iter);
end
set(gca, 'YScale', 'log'); grid on; legend(methods)
xlabel('iteration'); ylabel('residual')
title('Three iterative methods on the same 41 x 41 problem')

% The Omega Scan on Two Grids
% The handout asks for a scan from 1.0 to 1.99, repeated on a grid twice
% as fine. The sample points are spaced coarsely at the flat low end and
% densely near the top, where the sharp minimum and the blowup past it
% both live. The tolerance is relaxed to 1e-4, the scan is after the
% location of the minimum and that location does not move with the depth
% of the solve.
figure; hold on
for n = [31 61]
    Gn = em.solve.grid2d([0 1], [0 1], n, n);
    mk = false(n,n); mk(1,:)=true; mk(end,:)=true; mk(:,1)=true;
mk(:,end)=true;
    om = [1.0 1.4 1.7 1.85 1.90 1.95 1.99];
    it = zeros(size(om));
    for j = 1:numel(om)
        [~, ij] = em.solve.poisson2d(Gn, rho, 1, mk, zeros(n,n), ...
            struct('method','sor','tol',1e-4,'maxit',20000,'omega',om(j)));
        it(j) = ij.iter;
    end
    plot(om, it, 'o-')
    [~, jb] = min(it);
    fprintf('n = %3d   best omega about %.2f   theory 2/(1+sin(pi h)) =
%.3f\n', ...
        n, om(jb), 2/(1+sin(pi/(n-1))));
end
grid on; legend('31 x 31', '61 x 61')
xlabel('omega'); ylabel('iterations to tolerance')
title('SOR is sharp, the best omega moves with the grid')

% Timing, Iterative Against Direct
Nt = [41 61 81];
td = zeros(size(Nt)); ts = zeros(size(Nt));
for k = 1:numel(Nt)
```

```
    n = Nt(k);
    Gn = em.solve.grid2d([0 1], [0 1], n, n);
    mk = false(n,n); mk(1,:)=true; mk(end,:)=true; mk(:,1)=true;
mk(:,end)=true;
    tic; em.solve.poisson2d(Gn, rho, 1, mk, zeros(n,n),
struct('method','direct')); td(k) = toc;
    tic; em.solve.poisson2d(Gn, rho, 1, mk, zeros(n,n), ...
        struct('method','sor','tol',1e-6,'maxit',40000,'omega',2/(1+sin(pi/
(n-1))))); ts(k) = toc;
    fprintf('N = %3d   direct %.3f s   SOR %.3f s\n', n, td(k), ts(k));
end
figure; loglog(Nt, td, 'o-', Nt, ts, 's-'); grid on
xlabel('grid size N'); ylabel('wall time (s)')
legend('direct', 'SOR at optimal omega', 'location', 'northwest')
title('Cost against grid size')

% Carissa McWilliams, Mitch Nazareth, and Amelia Harris one or two
sentences. On grids this small the direct method wins
% outright. Name the two growth rates, and say at what problem size the
% trend claims the tables turn.

% The direct solver and SOR are two numerical methods with computational
% growth rates that describe how their runtimes increase as the total
% number of grid points, n, increases in a 2D system.
% A Laplace Sanity Pair
% One edge held at 1, the opposite at 0, Neumann on the other two. The
% answer must be a straight ramp, and the interior must hold no extremum.

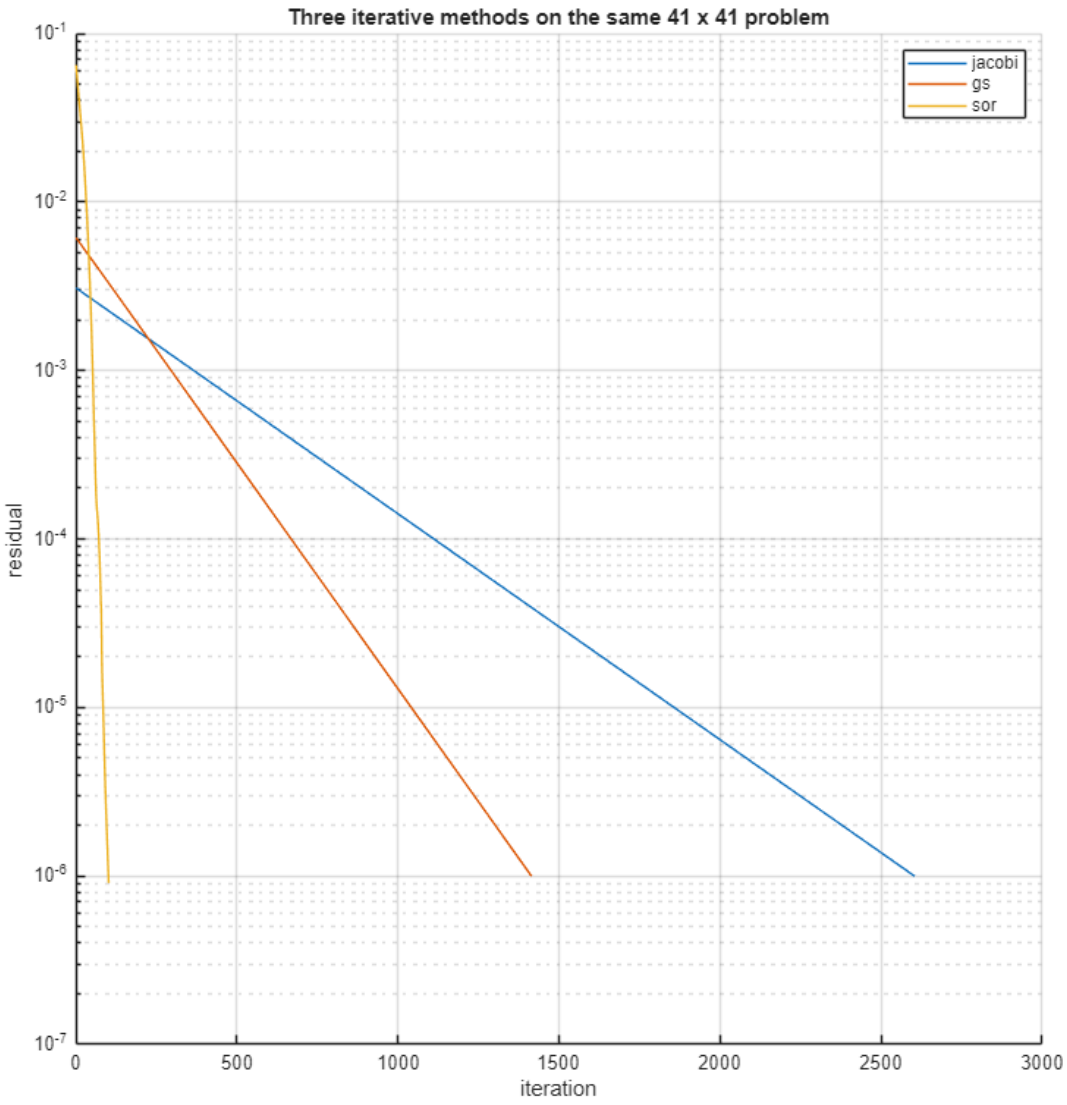
n = 41;
Gn = em.solve.grid2d([0 1], [0 1], n, n);
mk = false(n,n); mk(:,1) = true; mk(:,end) = true;
Vf = zeros(n,n); Vf(:,1) = 1;
VL = em.solve.poisson2d(Gn, 0, 1, mk, Vf, struct('method','direct'));
ramp = repmat(linspace(1, 0, n), n, 1);
fprintf('max deviation from the linear ramp = %.3e\n', max(abs(VL(:) -
ramp(:))));
inner = VL(2:end-1, 2:end-1);
fprintf('interior range [%.4f, %.4f] inside boundary range [0, 1] = %d\n',
...
    min(inner(:)), max(inner(:)), min(inner(:)) >= 0 && max(inner(:)) <= 1);

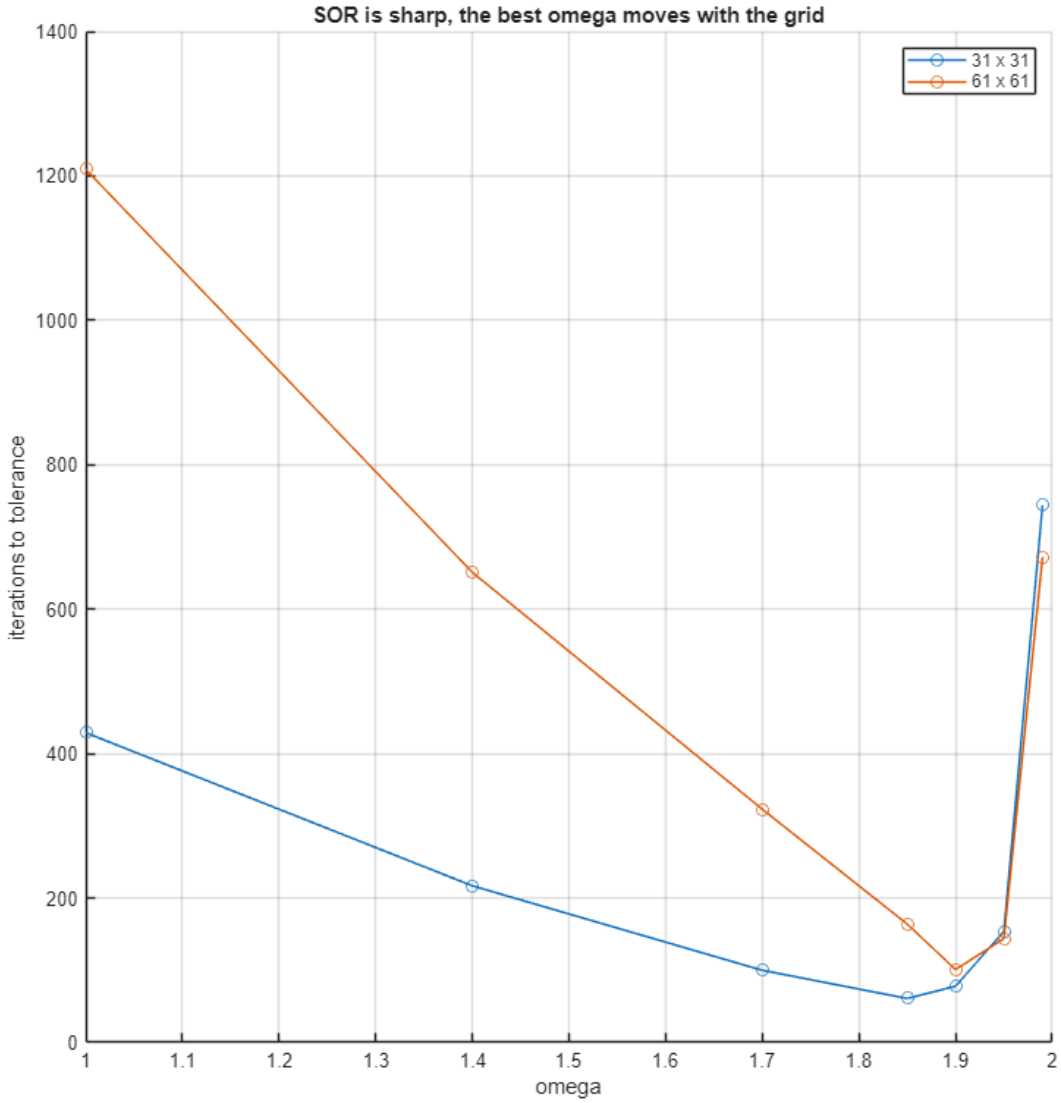
% Gauss's Law Closes the Loop
% A block of charge in a grounded box. The flux of E through a rectangle
% enclosing the block must equal the enclosed charge over eps0.
n = 121;
Gn = em.solve.grid2d([-1 1], [-1 1], n, n);
mk = false(n,n); mk(1,:)=true; mk(end,:)=true; mk(:,1)=true; mk(:,end)=true;
rhoB = @(X,Y) 1e-8 * (abs(X) < 0.25 & abs(Y) < 0.25);
VB = em.solve.poisson2d(Gn, rhoB, 1, mk, zeros(n,n),
struct('method','direct'));

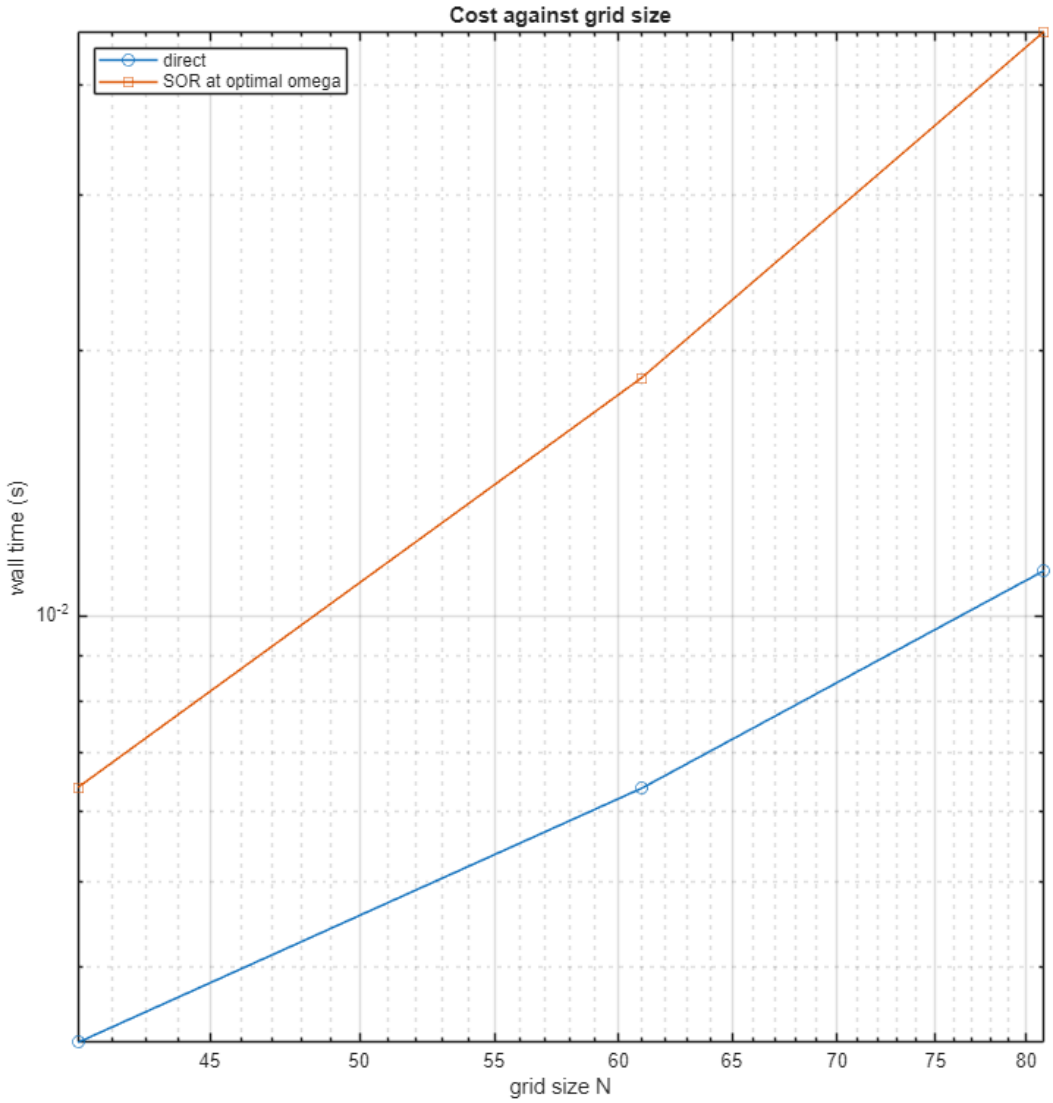
% Linear interpolation is used, Octave's spline path refuses scattered
% query points. The gradient step is half a cell so the difference reads
% the local slope of the interpolant.
```

```
Vh = @(r) interp2(Gn.X, Gn.Y, VB, r(:,1), r(:,2), 'linear');
Eh = @(r) -em.op.grad(Vh, r, Gn.hx/2);
edges = { @(t)[t -0.6+0*t 0*t], [0 -1 0]; @(t)[0.6+0*t t 0*t], [1 0 0]; ...
          @(t)[t 0.6+0*t 0*t], [0 1 0];  @(t)[-0.6+0*t t 0*t], [-1 0 0] };
flux = 0;
for k = 1:4
    [pos, w] = em.quad.line(edges{k,1}, [-0.6 0.6], 200);
    En = Eh(pos) * edges{k,2}';
    flux = flux + sum(En .* w);
end
Qenc = 1e-8 * 0.5 * 0.5;
fprintf('flux of E through the rectangle = %.4e\n', flux);
fprintf('enclosed charge over eps0       = %.4e\n', Qenc/e0);
fprintf('relative difference             = %.2e\n', abs(flux - Qenc/e0)/
(Qenc/e0));
```

```
MMS max error on the 81 x 81 grid = 1.285e-04
N =  21   max error = 2.059e-03
N =  41   max error = 5.142e-04
N =  81   max error = 1.285e-04
N = 161   max error = 3.212e-05
observed order p = 2.04  (expect near 2 for the five point stencil)
jacobi  iterations =  2604
gs      iterations =  1415
sor     iterations =   102
n =  31   best omega about 1.85   theory 2/(1+sin(pi h)) = 1.811
n =  61   best omega about 1.90   theory 2/(1+sin(pi h)) = 1.901
N =  41   direct 0.003 s   SOR 0.006 s
N =  61   direct 0.006 s   SOR 0.019 s
N =  81   direct 0.011 s   SOR 0.046 s
max deviation from the linear ramp = 7.772e-16
interior range [0.0250, 0.9750] inside boundary range [0, 1] = 1
flux of E through the rectangle = 2.6384e+02
enclosed charge over eps0       = 2.8235e+02
relative difference             = 6.56e-02
```


Three iterative methods on the same 41 x 41 problem
jacobi
gs
sor
residual
iteration
10^-1
10^-2
10^-3
10^-4
10^-5
10^-6
10^-7
0
500
1000
1500
2000
2500
3000


SOR is sharp, the best omega moves with the grid
31 x 31
61 x 61
iterations to tolerance
omega
0
200
400
600
800
1000
1200
1400
1
1.1
1.2
1.3
1.4
1.5
1.6
1.7
1.8
1.9
2



Interpretation

Carissa McWilliams, Mitch Nazareth, and Amelia Harris three to six sentences. The manufactured solution was reproduced and the observed order matched the stencil. Say what an observed order of 1 would have told you, and name the mistake that classically causes it.

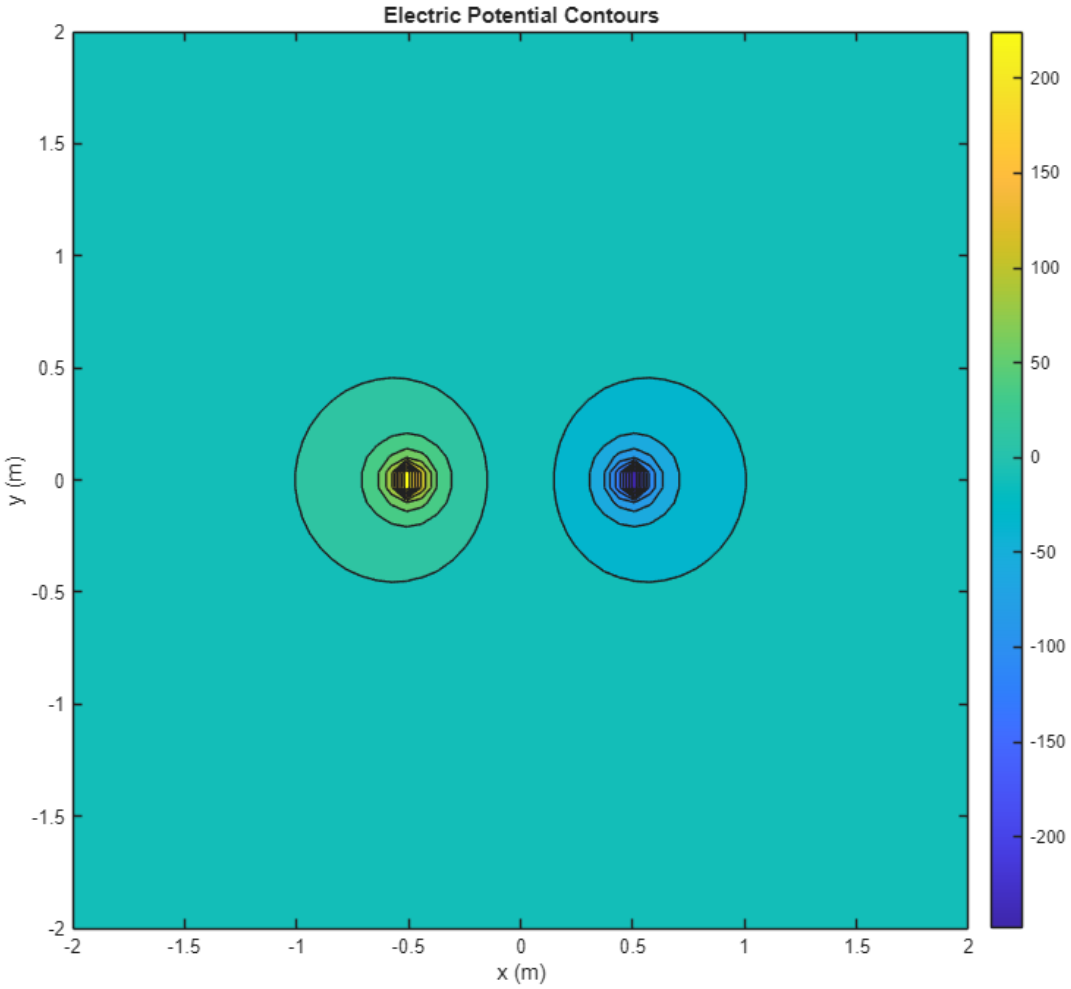
```
% An observed order of 1 would indicate first-order accuracy, meaning the
numerical error decreases by
% approximately half when the grid spacing is halved. This commonly occurs
% when a first-order approximation is used at the boundaries instead of a
% second-order approximation, reducing the overall convergence rate.
% Therefore, our results demonstrate that the Poisson solver accurately
% approximates the potential as the grid resolution increases.
```

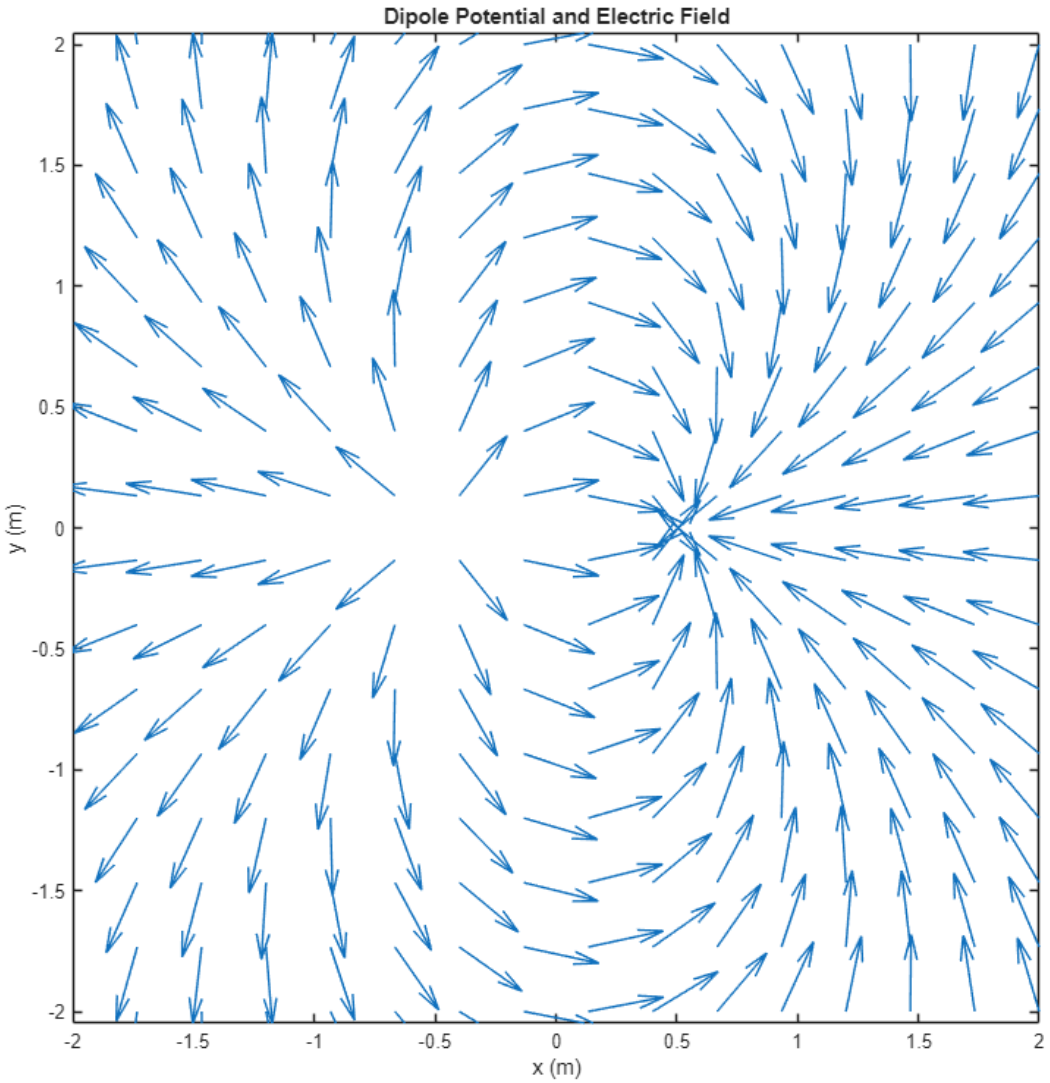
```
% Problems Encountered
% Carissa McWilliams, Mitch Nazareth, and Amelia Harris honest account, or
NONE.
% NONE
```

Full test suite

```
runTests
```

```
EECE 306 toolbox test suite
------------------------------------------------------------
  test_lab01 checks complete
  PASS  test_lab01            0.02 s
  PASS  test_lab02            0.02 s
  PASS  test_lab03            0.01 s
  PASS  test_lab04            0.01 s
  PASS  test_lab05            0.08 s
Test 1 passed: Point charge potential
Additional test passed: Superposition
Test 2 passed: E = -grad(V)
Source 1, Path 1: C = 1.632727e-11
Source 1, Path 2: C = -7.748910e-11
Source 1, Path 3: C = -3.812791e-11
Source 2, Path 1: C = 4.072621e-12
Source 2, Path 2: C = 1.249403e-12
Source 2, Path 3: C = -9.679293e-12
Source 3, Path 1: C = 7.818250e-10
Source 3, Path 2: C = 2.382805e-10
Source 3, Path 3: C = -1.847925e-09
Test 3 passed: Nine closed-path circulations
Test 4 passed: Non-conservative field
Test 5 passed: Potential difference
Warning: Source extent grows with element count. Potential referenced to
infinity may diverge.
Test 6 passed: Growing-source warning
Visualization completed
All six Lab 6 tests passed!
  PASS  test_lab06            0.38 s
Test 1 passed: MMS error = 8.225e-05
Test 2 passed: convergence order = 2.002
Test 3 passed: Jacobi, GS, SOR, direct agree
Test 4 passed: iterations SOR=118, GS=782, Jacobi=1505
Test 5 passed: mixed BC linear profile error 7.772e-16
Test 6 passed: no source-free interior extrema beyond BC bounds
Test 7 passed: Gauss law relative error = 1.128e-02
All seven Lab 7 tests passed!
  PASS  test_lab07            0.09 s
------------------------------------------------------------
  7 passed, 0 failed, 7 total
  All tests passing.
```


Electric Potential Contours
y (m)
x (m)
2
1.5
1
0.5
0
-0.5
-1
-1.5
-2
-2
-1.5
-1
-0.5
0
0.5
1
1.5
2
200
150
100
50
0
-50
-100
-150
-200



Published with MATLAB® R2025a